Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 1 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

Contents

1 Introduction **2**

2 Problem Definition **2**

3 Need for the Project **3**

4 Existing System **3**

5 Proposed System **3**
5.1 System Workflow 4

6 Stakeholders **4**

7 Project Objectives **4**
7.1 Functional Objectives 5
7.2 DevOps Objectives 5

8 Project Constraints **5**

9 Success Criteria **5**

10 Scope of the Project **6**
10.1 In Scope 6
10.2 Out of Scope 7

11 System Requirements **7**
11.1 Inventory Management Module 7
11.2 Stock Transaction Module 7
11.3 Dashboard Module 8
11.4 Search and Filter Module 8
11.5 Alert Module 8

12 Assumptions and Risks **8**
12.1 Assumptions 8
12.2 Risks and Mitigation 9

13 Project Roadmap **9**

14 Expected Deliverables **10**

15 Conclusion **11**

1. Introduction

Inventory management plays an important role in the smooth operation of every grocery store. It helps shop owners keep track of available products, monitor stock levels, and ensure that essential items are always available for customers. Proper inventory management reduces wastage, prevents stock shortages, and improves overall business efficiency.

Many small and medium-sized grocery stores still depend on manual methods such as paper registers or spreadsheet software to manage inventory. Although these methods are inexpensive, they often result in inaccurate records, duplicate entries, delayed stock updates, and difficulty in finding product information. As the number of products increases, managing inventory manually becomes more challenging and time-consuming.

The **Automated Grocery Inventory Dashboard** is a web-based application designed to simplify inventory management. It provides a centralized dashboard where users can manage grocery items, update stock levels, record stock transactions, search products, and receive alerts when inventory becomes low.

Apart from developing the application, this project also focuses on implementing modern DevOps practices. Throughout the 15-week project timeline, tools such as Git, GitHub, Jenkins, Selenium, Docker, and Ansible will be integrated to automate software development, testing, deployment, and infrastructure management.

2. Problem Definition

Many grocery stores continue to manage their inventory using notebooks, Excel sheets, or other manual methods. These approaches require continuous manual updates and make it difficult to maintain accurate inventory records. Human errors during stock entry often lead to incorrect inventory information, resulting in either overstocking or stock shortages.

Store owners also face difficulties in identifying products that are running low because there is no automated monitoring system. Searching for a particular product or reviewing stock availability becomes time-consuming, especially when dealing with a large number of items.

The lack of a centralized inventory management system affects decision-making, increases workload, and reduces overall operational efficiency.

Therefore, there is a need for a simple and automated inventory management solution that provides real-time stock monitoring, easy product management, and timely stock alerts while demonstrating modern DevOps practices throughout the software development lifecycle.

3. Need for the Project

Managing inventory manually becomes difficult as the number of products in a grocery store increases. Store owners need an efficient system that provides accurate stock information without maintaining paper records or spreadsheets. A centralized dashboard helps users update inventory quickly, search products easily, and monitor stock availability in real time.

The proposed system reduces manual effort, minimizes human errors, and ensures that important products are always available by generating low-stock alerts. It also serves as a practical project to implement DevOps concepts such as version control, continuous integration, automated testing, containerization, and automated deployment throughout the software development lifecycle.

4. Existing System

Most small grocery stores manage their inventory manually using notebooks or spreadsheet applications. Whenever new stock arrives or products are sold, the inventory records are updated manually. Employees also have to physically verify stock availability to identify products that need to be reordered.

Although this approach works for a small number of products, it becomes inefficient as the inventory grows. Maintaining accurate records requires significant manual effort and increases the chances of mistakes.

5. Proposed System

The proposed **Automated Grocery Inventory Dashboard** is a web-based application that simplifies inventory management by providing a centralized platform for maintaining grocery stock.

The system allows authorized users to manage grocery items, record stock transactions, search products, and monitor inventory through an interactive dashboard. Every item stored in the inventory contains essential details such as product name, category, quantity, unit price, available stock, and reorder threshold.

Whenever new stock is received, the quantity can be updated using the **Stock In** feature. Similarly, when products are sold or removed from inventory, the **Stock Out** feature automatically updates the available quantity. The dashboard continuously monitors inventory levels and displays alerts whenever the stock falls below the predefined threshold or becomes unavailable.

The application also provides search and filter options that allow users to quickly locate products based on their name or category.

This project follows the Minimum Viable Product (MVP) approach by implementing only the essential inventory management features. During the later stages of the project,

DevOps tools such as Git, Jenkins, Selenium, Docker, and Ansible will be integrated to automate software development, testing, deployment, and infrastructure provisioning.

5.1 System Workflow

Store Staff / Admin
↓
Manage Grocery Items
(Add / Update / Delete)
↓
Record Stock Transactions
(Stock In / Stock Out)
↓
Inventory Database
↓
Dashboard
(Total Items, Search, Filter, Low Stock Alerts)

6. Stakeholders

The successful operation of the Grocery Inventory Dashboard depends on different users who interact with the system. Their responsibilities are summarized below.

Stakeholder	**Responsibility**
Store Owner	Monitor inventory, analyze stock availability, and make purchasing decisions.
Inventory Manager	Manage grocery items and maintain stock records.
Store Staff	Perform stock-in and stock-out transactions and update inventory.
System Administrator	Maintain the application, database, and deployment environment.
Development Team	Develop and enhance the application features.
DevOps Engineer	Implement CI/CD pipeline, testing, deployment, and automation.

7. Project Objectives

The primary objective of this project is to develop a simple and efficient web-based Grocery Inventory Dashboard that helps users manage inventory digitally while demonstrating the complete DevOps lifecycle.

7.1 Functional Objectives

- Develop a centralized inventory management system for grocery products.
- Allow users to add, update, delete, and view grocery items.
- Maintain accurate stock levels using Stock In and Stock Out transactions.
- Provide search and filter functionality for quick product lookup.
- Display inventory statistics through an interactive dashboard.
- Generate alerts for low-stock and out-of-stock products.

7.2 DevOps Objectives

- Manage source code using Git and GitHub.
- Implement Continuous Integration (CI) using Jenkins.
- Automate testing using Selenium WebDriver.
- Containerize the application using Docker.
- Automate deployment and provisioning using Ansible.
- Demonstrate a complete CI/CD pipeline from development to deployment.

8. Project Constraints

The project will be developed within the constraints specified below.

Constraint	**Description**
Project Duration	15 Weeks
Team Size	Individual Project
Budget	Only free and open-source software will be used.
Development Platform	Personal laptop and local development environment.
Deployment	Local deployment using Docker containers.
Technology Stack	Java, Spring Boot, Maven, MySQL, Git, Jenkins, Selenium, Docker and Ansible.
Project Scope	Limited to the Minimum Viable Product (MVP).

9. Success Criteria

The project will be considered successful if the following objectives are achieved.

Success Metric	Expected Outcome
Inventory Management	Items can be added, updated, deleted and viewed successfully.
Stock Transactions	Stock In and Stock Out update inventory correctly.
Search Functionality	Users can quickly search and filter products.
Inventory Alerts	Low-stock and out-of-stock alerts are generated correctly.
Dashboard	Displays accurate inventory statistics.
Jenkins Build	Application builds successfully through Jenkins.
Automated Testing	Selenium test cases execute successfully.
Docker Deployment	Application runs successfully inside a Docker container.
Ansible Automation	Target environment is configured successfully using Ansible.

10. Scope of the Project

The project follows the Minimum Viable Product (MVP) approach by implementing only the core inventory management functionalities required for demonstration.

10.1 In Scope

- Inventory Item Management
- Add, Update and Delete Grocery Items
- Stock In and Stock Out Transactions
- Inventory Dashboard
- Product Search and Category Filter
- Low Stock and Out-of-Stock Alerts
- Git Version Control
- Jenkins Continuous Integration
- Selenium Automated Testing
- Docker Containerization
- Ansible-based Deployment Automation

10.2 Out of Scope

- Online Billing or Point of Sale (POS)
- Barcode or QR Code Scanner
- Supplier Management
- Customer Management
- Online Payment Integration
- Mobile Application
- Cloud Deployment
- AI-based Demand Forecasting
- Email or SMS Notifications
- Multi-store Inventory Management

11. System Requirements

The Automated Grocery Inventory Dashboard is designed as a simple web-based application that helps grocery stores maintain inventory digitally. The system focuses on core inventory operations and provides users with a centralized dashboard for monitoring stock availability.

The application is divided into the following functional modules.

11.1 Inventory Management Module

This module manages the complete grocery item catalogue. Users can add new products, update existing product details, delete products that are no longer available, and view the complete inventory list.

Each inventory item stores information such as:

- Product ID
- Product Name
- Category
- Unit Price
- Available Quantity
- Reorder Level

This module forms the foundation of the entire application since all inventory operations are performed on these records.

11.2 Stock Transaction Module

The Stock Transaction module keeps the inventory updated whenever products are added or removed.

It consists of two main operations:

- **Stock In** – Increases the available quantity whenever new stock is received.

- **Stock Out** – Decreases the available quantity whenever products are sold or removed.

Every transaction immediately updates the available inventory so that the dashboard always displays the latest stock information.

11.3 Dashboard Module

The dashboard provides a quick overview of the inventory without requiring users to inspect individual products.

The dashboard displays:

- Total Products
- Total Categories
- Low Stock Products
- Out-of-Stock Products

This helps users monitor inventory health and identify products that require immediate attention.

11.4 Search and Filter Module

As the number of products increases, locating a particular item manually becomes difficult.

This module allows users to:

- Search products by name.
- Filter products based on category.

The search functionality improves usability and reduces the time required to find inventory items.

11.5 Alert Module

The Alert module continuously monitors the available quantity of every product.

If the available stock becomes less than the predefined reorder level, the system generates a Low Stock Alert. If the quantity becomes zero, the product is marked as Out of Stock.

These alerts help users reorder products before inventory shortages occur.

12. Assumptions and Risks

12.1 Assumptions

The project is developed based on the following assumptions.

- Users have basic knowledge of operating a web application.

- Grocery inventory data is entered correctly by authorized users.
- A local MySQL database is available for storing inventory records.
- Required software such as Git, Java, Maven, Docker, Jenkins and Ansible can be installed successfully.
- The application will initially be deployed in a local development environment.

12.2 Risks and Mitigation

Risk	**Mitigation Strategy**
Delay in implementation	Follow the weekly project milestones and complete tasks on schedule.
Git merge conflicts	Use feature branches and perform regular commits.
Tool installation issues	Verify installation and compatibility before development begins.
Build or deployment failures	Test the application locally before integrating the CI/CD pipeline.
Configuration errors	Use Ansible automation to maintain consistent system configuration.

13. Project Roadmap

The project will be developed incrementally over 15 weeks following the DevOps lifecycle. Each week focuses on a specific phase, gradually building the application and integrating DevOps tools.

Week	Major Activity
1	Problem Definition and Project Scope
2	Agile Planning, Product Backlog and DevOps Workflow
3	Requirements Analysis, System Design and Architecture
4	GitHub Repository Setup and Version Control
5	Development of First Core Feature using Git Branching
6	Completion of MVP and Git Collaboration
7	Jenkins Installation and Continuous Integration
8	Pipeline as Code and Application Deployment
9	Selenium Test Development and Execution
10	Continuous Testing using Jenkins Pipeline
11	Docker Image Creation and Container Management
12	Jenkins-Docker Continuous Deployment Pipeline
13	Configuration Management using Ansible
14	Automated Provisioning and Reliability Validation
15	Final Demonstration, Documentation and Viva

This roadmap ensures that the project is completed in a structured manner while progressively implementing DevOps practices from development to deployment.

14. Expected Deliverables

The following deliverables are expected from the successful completion of the project over the 15-week development timeline.

Deliverable	Description
Web Application	A functional Grocery Inventory Dashboard with essential inventory management features.
Source Code Repository	Well-organized source code managed using Git and GitHub.
CI/CD Pipeline	Automated build, testing, and deployment pipeline using Jenkins.
Automated Test Suite	Selenium test cases for validating key application functionalities.
Containerized Application	Dockerized application for consistent deployment across environments.
Configuration Automation	Ansible playbook for automated provisioning and deployment.
Project Documentation	Complete project report, architecture diagrams, screenshots, and technical documentation.
Presentation	Final project presentation and demonstration of the complete DevOps workflow.

15. Conclusion

The Automated Grocery Inventory Dashboard aims to provide a simple and efficient solution for managing grocery inventory through a centralized web application. The proposed system replaces manual inventory tracking with digital record management, making it easier to maintain stock information, monitor inventory levels, and identify products that require replenishment.

The project follows the Minimum Viable Product (MVP) approach by focusing only on the essential inventory management features required for demonstration. Along with application development, the project also emphasizes the practical implementation of DevOps practices including version control, continuous integration, automated testing, containerization, and automated deployment.

The work completed in Week 1 establishes the project scope, identifies the system requirements, defines the stakeholders, and provides a clear development roadmap for the remaining weeks. These deliverables serve as the foundation for implementing the complete end-to-end DevOps workflow in the upcoming phases of the project.